

使用 Service 连接到应用

使用 Service 连接到应用

Kubernetes 连接容器的模型 {#the-kubernetes-model-for-connecting-containers}

既然有了一个持续运行、可复制的应用，我们就能够将它暴露到网络上。

Kubernetes 假设 Pod 可与其它 Pod 通信，不管它们在哪个主机上。Kubernetes 给每一个 Pod 分配一个集群私有 IP 地址，所以没必要在 Pod 与 Pod 之间创建连接或将容器的端口映射到主机端口。这意味着同一个 Pod 内的所有容器能通过 localhost 上的端口互相连通，集群中的所有 Pod 也不需要通过 NAT 转换就能够互相看到。本文档的剩余部分详述如何在上述网络模型之上运行可靠的服务。

本教程使用一个简单的 Nginx 服务器来演示概念验证原型。

在集群中暴露 Pod {#exposing-pods-to-the-cluster}

我们在之前的示例中已经做过，然而让我们以网络连接的视角再重做一遍。创建一个 Nginx Pod，注意其中包含一个容器端口的规约：

这使得可以从集群中任何一个节点来访问它。检查节点，该 Pod 正在运行：

```
kubectl apply -f ./run-my-nginx.yaml
kubectl get pods -l run=my-nginx -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
my-nginx-3800858182-jr4a2	1/1	Running	0	13s	10.244.3.4	kubernetes-minion-905m
my-nginx-3800858182-kna2y	1/1	Running	0	13s	10.244.2.5	kubernetes-minion-ljyd

检查 Pod 的 IP 地址：

```
kubectl get pods -l run=my-nginx -o custom-columns=POD_IP:.status.podIPs
POD_IP
[map[ip:10.244.3.4]]
[map[ip:10.244.2.5]]
```

你应该能够通过 ssh 登录到集群中的任何一个节点上，并使用诸如 curl 之类的工具向这两个 IP 地址发出查询请求。需要注意的是，容器 **不会** 使用该节点上的 80 端口，也不会使用任何特定的 NAT 规则去路由流量到 Pod 上。这意味着你可以使用相同的 containerPort 在同一个节点上运行多个 Nginx Pod，并且可以从集群中任何其他的 Pod 或节点上使用为 Pod 分配的 IP 地址访问到它们。如果你想的话，你依然可以将宿主节点的某个端口的流量转发到 Pod 中，但是出于网络模型的原因，你不必这么做。

如果对此好奇，请参考 Kubernetes 网络模型。

创建 Service {#creating-a-service}

我们有一组在一个扁平的、集群范围的地址空间中运行 Nginx 服务的 Pod。理论上，你可以直接连接到这些 Pod，但如果某个节点宕机会发生什么呢？Pod 会终止，Deployment 内的 ReplicaSet 将创建新的 Pod，且使用不同的 IP。这正是 Service 要解决的问题。

Kubernetes Service 是集群中提供相同功能的一组 Pod 的抽象表达。当每个 Service 创建时，会被分配一个唯一的 IP 地址（也称为 clusterIP）。这个 IP 地址与 Service 的生命周期绑定在一起，只要 Service 存在，它就不会改变。可以配置 Pod 使它与 Service 进行通信，Pod 知道与 Service 通信将被自动地负载均衡到该 Service 中的某些 Pod 上。

可以使用 kubectl expose 命令为 2 个 Nginx 副本创建一个 Service：

```
kubectl expose deployment/my-nginx  
service/my-nginx exposed
```

这等价于使用 kubectl create -f 命令及如下的 yaml 文件创建：

上述规约将创建一个 Service，该 Service 会将所有具有标签 run: my-nginx 的 Pod 的 TCP 80 端口暴露到一个抽象的 Service 端口上（targetPort：容器接收流量的端口；port：可任意取值的抽象的 Service 端口，其他 Pod 通过该端口访问 Service）。查看 Service API 对象以了解 Service 所能接受的字段列表。查看你的 Service 资源：

```
kubectl get svc my-nginx
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-nginx	ClusterIP	10.0.162.149		80/TCP	21s

正如前面所提到的，一个 Service 由一组 Pod 提供支撑。这些 Pod 通过 暴露出来。Service Selector 将持续评估，结果被 POST 到使用与该 Service 连接的一个

EndpointSlice。当 Pod 终止后，它会自动从包含该 Pod 的 EndpointSlices 中移除。新的能够匹配上 Service Selector 的 Pod 将被自动地为该 Service 添加到 EndpointSlice 中。检查 Endpoint，注意到 IP 地址与在第一步创建的 Pod 是相同的。

```
kubectl describe svc my-nginx
```

```
Name:          my-nginx
Namespace:     default
Labels:        run=my-nginx
Annotations:
Selector:      run=my-nginx
Type:          ClusterIP
IP Family Policy: SingleStack
IP Families:   IPv4
IP:            10.0.162.149
IPs:           10.0.162.149
Port:          80/TCP
TargetPort:    80/TCP
Endpoints:     10.244.2.5:80,10.244.3.4:80
Session Affinity: None
Events:
```

```
kubectl get endpointslices -l kubernetes.io/service-name=my-nginx
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
my-nginx-7vzhx	IPv4	80	10.244.2.5,10.244.3.4	21s

现在，你应该能够从集群中任意节点上使用 curl 命令向 : 发送请求以访问 Nginx Service。注意 Service IP 完全是虚拟的，它从来没有走过网络，如果对它如何工作的原理感到好奇，可以进一步阅读服务代理的内容。

访问 Service {#accessing-the-service}

Kubernetes 支持两种查找服务的主要模式：环境变量和 DNS。前者开箱即用，而后者则需要 CoreDNS 集群插件。

如果不需要服务环境变量（因为可能与预期的程序冲突，可能要处理的变量太多，或者仅使用 DNS 等），则可以通过在 pod spec 上将 enableServiceLinks 标志设置为 false 来禁用此模式。

环境变量 {#environment-variables}

当 Pod 在节点上运行时，kubelet 会针对每个活跃的 Service 为 Pod 添加一组环境变量。这就引入了一个顺序的问题。为解释这个问题，让我们先检查正在运行的 Nginx Pod 的环境变量（你的环境中的 Pod 名称将会与下面示例命令中的不同）：

```
kubectl exec my-nginx-3800858182-jr4a2 -- printenv | grep SERVICE
```

```
KUBERNETES_SERVICE_HOST=10.0.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
```

能看到环境变量中并没有你创建的 Service 相关的值。这是因为副本的创建先于 Service。这样做的另一个缺点是，调度器可能会将所有 Pod 部署到同一台机器上，如果该机器宕机则整个 Service 都会离线。要改正的话，我们可以先终止这 2 个 Pod，然后等待 Deployment 去重新创建它们。这次 Service 会**先于**副本存在。这将实现调度器级别的 Pod 按 Service 分布（假定所有的节点都具有同样的容量），并提供正确的环境变量：

```
kubectl scale deployment my-nginx --replicas=0; kubectl scale deployment my-nginx --replicas=2;
```

```
kubectl get pods -l run=my-nginx -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
my-nginx-3800858182-e9ihh	1/1	Running	0	5s	10.244.2.7	kubernetes-minion-ljyd
my-nginx-3800858182-j4rm4	1/1	Running	0	5s	10.244.3.8	kubernetes-minion-905m

你可能注意到，Pod 具有不同的名称，这是因为它们是被重新创建的。

```
kubectl exec my-nginx-3800858182-e9ihh -- printenv | grep SERVICE
```

```
KUBERNETES_SERVICE_PORT=443
MY_NGINX_SERVICE_HOST=10.0.162.149
KUBERNETES_SERVICE_HOST=10.0.0.1
MY_NGINX_SERVICE_PORT=80
KUBERNETES_SERVICE_PORT_HTTPS=443
```

DNS

Kubernetes 提供了一个自动为其它 Service 分配 DNS 名字的 DNS 插件 Service。你可以通过如下命令检查它是否在工作：

```
kubectl get services kube-dns --namespace=kube-system
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kube-dns	ClusterIP	10.0.0.10		53/UDP,53/TCP	8m

本段剩余的内容假设你已经有一个拥有持久 IP 地址的 Service（my-nginx），以及一个为其 IP 分配名称的 DNS 服务器。这里我们使用 CoreDNS 集群插件（应用名为 kube-dns），所以在集群中的任何 Pod 中，你都可以使用标准方法（例如：gethostbyname()）与该 Service 通信。如果 CoreDNS 没有在运行，你可以参照 CoreDNS README 或者安装 CoreDNS 来启用它。让我们运行另一个 curl 应用来进行测试：

```
kubectl run curl --image=radial/busyboxplus:curl -i --tty --rm
```

```
Waiting for pod default/curl-131556218-9fnch to be running, status is Pending, pod ready: false
```

```
Hit enter for command prompt
```

然后，按回车并执行命令 nslookup my-nginx：

```
[ root@curl-131556218-9fnch:/ ]$ nslookup my-nginx
```

```
Server: 10.0.0.10
```

```
Address 1: 10.0.0.10
```

```
Name: my-nginx
```

```
Address 1: 10.0.162.149
```

保护 Service {#securing-the-service}

到现在为止，我们只在集群内部访问了 Nginx 服务器。在将 Service 暴露到因特网之前，我们希望确保通信信道是安全的。为实现这一目的，需要：

- 用于 HTTPS 的自签名证书（除非已经有了一个身份证书）
- 使用证书配置的 Nginx 服务器
- 使 Pod 可以访问证书的 Secret

你可以从 Nginx https 示例获取所有上述内容。你需要安装 go 和 make 工具。如果你不想安装这些软件，可以按照后文所述的手动执行步骤执行操作。简要过程如下：

```
make keys KEY=/tmp/nginx.key CERT=/tmp/nginx.crt
```

```
kubectl create secret tls nginxsecret --key /tmp/nginx.key --cert /tmp/nginx.crt
```

```
secret/nginxsecret created
```

```
kubectl get secrets
```

NAME	TYPE	DATA	AGE
nginxsecret	kubernetes.io/tls		2 1m

以下是 ConfigMap：

```
kubectl create configmap nginxconfigmap --from-file=default.conf
```

你可以在 Kubernetes examples 项目代码仓库中找到 default.conf 示例。

```
configmap/nginxconfigmap created
```

```
kubectl get configmaps
```

NAME	DATA	AGE
nginxconfigmap	1	114s

你可以使用以下命令来查看 nginxconfigmap ConfigMap 的细节：

```
kubectl describe configmap nginxconfigmap
```

输出类似于：

Name: nginxconfigmap

Namespace: default

Labels:

Annotations:

Data

====

default.conf:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server ipv6only=on;

    listen 443 ssl;

    root /usr/share/nginx/html;
    index index.html;

    server_name localhost;
    ssl_certificate /etc/nginx/ssl/tls.crt;
```

```

    ssl_certificate_key /etc/nginx/ssl/tls.key;

    location / {
        try_files $uri $uri/ =404;
    }
}

```

BinaryData
====

Events:

以下是你在运行 make 时遇到问题时要遵循的手动步骤（例如，在 Windows 上）：

```

# 创建公钥和相对应的私钥
openssl req -x509 -noenc -days 365 -newkey rsa:2048 -keyout /d/tmp/nginx.key -out
/d/tmp/nginx.crt -subj "/CN=my-nginx/O=my-nginx"
# 对密钥实施 base64 编码
cat /d/tmp/nginx.crt | base64 | tr -d '\n'
cat /d/tmp/nginx.key | base64 | tr -d '\n'

```

如下所示，使用上述命令的输出创建 yaml 文件。base64 编码的值应全部放在一行上。

```

apiVersion: "v1"
kind: "Secret"
metadata:
  name: "nginxsecret"
  namespace: "default"
type: kubernetes.io/tls
data:
  # 注意：将以下值替换为你自己 base64 编码后的证书和密钥。
  tls.crt: "REPLACE_WITH_BASE64_CERT"
  tls.key: "REPLACE_WITH_BASE64_KEY"

```

现在使用文件创建 Secret：

```

kubectl apply -f nginxsecrets.yaml
kubectl get secrets

```

NAME	TYPE	DATA	AGE
nginxsecret	kubernetes.io/tls		2 1m

现在修改 Nginx 副本以启动一个使用 Secret 中的证书的 HTTPS 服务器以及相应的用于暴露其端口（80 和 443）的 Service：

关于 nginx-secure-app 清单，值得注意的几点如下：

- 它将 Deployment 和 Service 的规约放在了同一个文件中。
- Nginx 服务器通过 80 端口处理 HTTP 流量，通过 443 端口处理 HTTPS 流量，而 Nginx Service 则暴露了这两个端口。
- 每个容器能通过挂载在 /etc/nginx/ssl 的卷访问密钥。卷和密钥需要在 Nginx 服务器启动之前配置好。

```
kubectl delete deployments,svc my-nginx; kubectl create -f ./nginx-secure-app.yaml
```

这时，你可以从任何节点访问到 Nginx 服务器。

```
kubectl get pods -l run=my-nginx -o custom-columns=POD_IP:.status.podIPs
POD_IP
[map[ip:10.244.3.5]]
```

```
node $ curl -k https://10.244.3.5
```

```
...
```

```
Welcome to nginx!
```

注意最后一步我们是如何提供 -k 参数执行 curl 命令的，这是因为在证书生成时，我们不知道任何关于运行 nginx 的 Pod 的信息，所以不得不在执行 curl 命令时忽略 CName 不匹配的情况。通过创建 Service，我们连接了在证书中的 CName 与在 Service 查询时被 Pod 使用的实际 DNS 名字。让我们从一个 Pod 来测试（为了方便，这里使用同一个 Secret，Pod 仅需要使用 nginx.crt 去访问 Service）：

```
kubectl apply -f ./curlpod.yaml
kubectl get pods -l app=curlpod
```

```
NAME                                READY  STATUS   RESTARTS  AGE
curl-deployment-1515033274-1410r  1/1    Running  0         1m
```

```
kubectl exec curl-deployment-1515033274-1410r -- curl https://my-nginx --cacert
/etc/nginx/ssl/tls.crt
```

```
...
```

```
Welcome to nginx!
```

```
...
```


暴露 Service {#exposing-the-service}

对应用的某些部分，你可能希望将 Service 暴露在一个外部 IP 地址上。Kubernetes 支持两种实现方式：NodePort 和 LoadBalancer。在上一段创建的 Service 使用了 NodePort，因此，如果你的节点有一个公网 IP，那么 Nginx HTTPS 副本已经能够处理因特网上的流量。

```
kubectl get svc my-nginx -o yaml | grep nodePort -C 5
```

```
uid: 07191fb3-f61a-11e5-8ae5-42010af00002
spec:
  clusterIP: 10.0.162.149
  ports:
  - name: http
    nodePort: 31704
    port: 8080
    protocol: TCP
    targetPort: 80
  - name: https
    nodePort: 32453
    port: 443
    protocol: TCP
    targetPort: 443
  selector:
    run: my-nginx
```

```
kubectl get nodes -o yaml | grep ExternalIP -C 1
```

```
- address: 104.197.41.11
  type: ExternalIP
  allocatable:
--
- address: 23.251.152.56
  type: ExternalIP
  allocatable:
...
```

```
$ curl https://: -k
```

```
...
```

```
Welcome to nginx!
```

让我们重新创建一个 Service 以使用云负载均衡器。将 my-nginx Service 的 Type 由 NodePort 改成 LoadBalancer:

```
kubectl edit svc my-nginx
kubectl get svc my-nginx
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-nginx	LoadBalancer	10.0.162.149	xx.xxx.xxx.xxx	8080:30163/TCP	21s

```
curl https:// -k
```

...

Welcome to nginx!

在 EXTERNAL-IP 列中的 IP 地址能在公网上被访问到。CLUSTER-IP 只能从集群/私有云网络中访问。

注意，在 AWS 上，类型 LoadBalancer 的服务会创建一个 ELB，且 ELB 使用主机名（比较长），而不是 IP。ELB 的主机名太长以至于不能适配标准 `kubectl get svc` 的输出，所以需要通过执行 `kubectl describe service my-nginx` 命令来查看它。可以看到类似如下内容：

```
kubectl describe service my-nginx
```

...

```
LoadBalancer Ingress: a320587ffd19711e5a37606cf4a74574-1142138393.us-east-1.elb.amazonaws.com
```

...

- 进一步了解如何使用 Service 访问集群中的应用
- 进一步了解如何使用 Service 将前端连接到后端
- 进一步了解如何创建外部负载均衡器